Republic of Iraq
Ministry of Higher Education and
Scientific Research
University of Babylon
College of Science for Women
Department of Computer Science



Automating architectural modeling to increase productivity using (Dynamo) and (Revit)

Research submitted by the student

Saja Haider Abdul-Zahra

To the Council of the College of Science for Women, Department of Computer Science / University of Babylon, as part of the requirements for obtaining a bachelor's degree in Computer Science.

Supervised by

Dr. Sahar Adill Kadhum

2026 AD 1447 AH



(يَرْفَعِ اللهُ الَّذِينَ آمَنُوا مِنْكُمْ وَ الَّذِينَ أُوتُوا الْعِلْمَ دَرَجَاتٍ)

[المجادلة: 11]



Dedication

To the one who is absent yet ever-present... My pillar of strength who left me midway through the journey...

To my late Father (May his soul rest in peace)...

To you, who departed in body but remained in every detail of my soul and every spark of my motivation. I dedicate this achievement and my graduation to you. Rest in peace, Father; your daughter’s ambition did not break despite the bitterness of loss. I have completed the path guided by your memory and the values you instilled in me.

To my beloved Mother...

My true support and the pillar of my life. You carried the weight of this journey with me, being both mother and father, and encouraged me through every moment of despair. Your patience and prayers are what brought me to this day.

To my companions and friends...

To those who lightened the burden of the days and the long hours of study. You were the best sisters and the finest company throughout this journey of knowledge and life.

To everyone who ever believed in me... I dedicate this fruit of my labor.

Acknowledgments

I would like to express my deepest gratitude and appreciation to my supervisor, Dr. Sahar Adill Kadhum, for her invaluable guidance, constant encouragement, and insightful feedback throughout the development of this research. Her expertise and support were essential in overcoming the technical challenges and bringing this project to completion.

I also extend my thanks to the Department of Computer Science for providing the academic environment and resources that facilitated my learning and growth.

Finally, I am grateful to everyone who provided technical assistance or shared their knowledge, contributing to the success of this work.

Supervisor Certification

I certify that the dissertation entitled “Automating architectural modeling to increase productivity using (Dynamo) and (Revit)” was prepared under my supervision at the Department of Computer Science, by the student Saja Haider Abdul-Zahr as partial fulfillment of the requirements for the Graduation Research Project.

Signature:

Name: Dr. Sahar Adill Kadhum

Date: / / 2026

Content

1 **Chapter One:Introduction**

1.1 Overview of Architectural Modeling 11

1.2 Technologies Used in the Evaluation 11

1.3 Problem Statement 12

1.4 Research Objectives 12

1.5 Research Importance 13

2 **Chapter Two:Theory Part**

2.1 Introduction 15

2.2 Building Information Modeling (BIM) Concept 16

2.3 Algorithmic and Generative Design 17

2.4 Visual Programming Environment (Dynamo) & API Integration 18

2.5 Smart Infrastructure and Residential Complexes 19

3 **Chapter Three: Practical Part**

3.1 Introduction 21

3.2 Global Infrastructure & Grid Establishment 22

3.3 Automated Residential Cluster Generation 23

3.4 Procedural Urban Furnishing & Unit Details 24

3.5 Stochastic Landscaping & Advanced System UI 25

4 **Chapter Four: Results and Discussion**

4.1 Introduction 27

4.2 Infrastructure and Grid Execution 28

4.3 Automated Residential Units Generationt 29

4.4 Urban Furnishing and Site Detailing Execution 30

4.5 Comprehensive Final Model Visualization 31

5 **Chapter Five: Conclusions and Future Work**

5.1 Introduction 34

5.2 Full Project Workflow 35

5.3 Integrated Results & Visualization 36

5.4 Future Work & Development 37

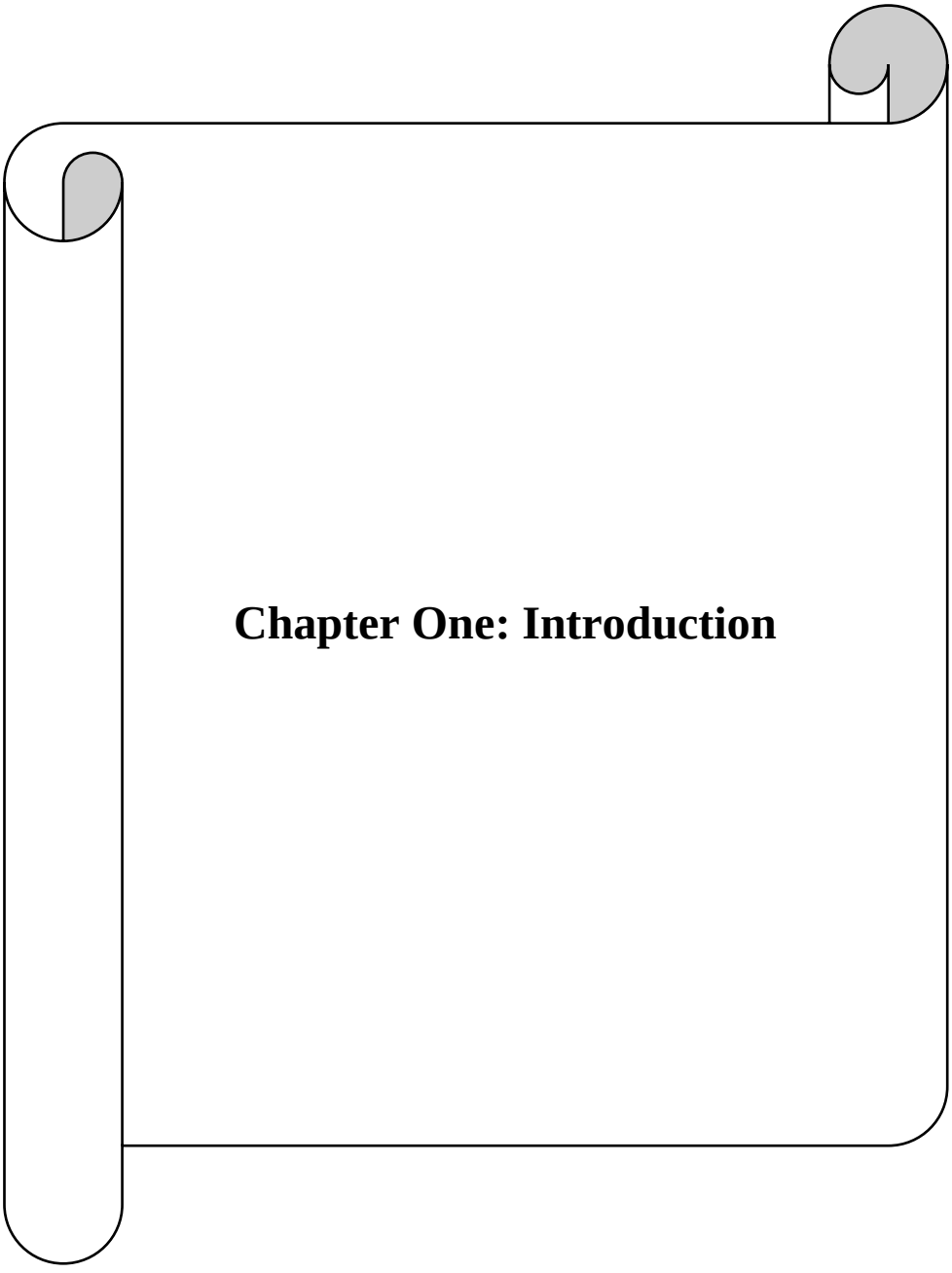
6 **References** 38

Table of Figures

(Fig-2-1: Automated BIM Modeling via Dynamo and Python)..... 15
(Fig-2-2: BIM Data-Driven Elements)..... 16
((Fig-2-3: Generative Model Flowchart) 17
((Fig-2-4: Dynamo & Python API Integration) 18
(Fig-2-5: Automated Security Bollard Component) 19
(fig-3-1: framework of the automated system)..... 21
(Code 3-2: Infrastructure & Grid Script)..... 22
(Code 3-3: Residential Units Logic) 23
(Code 3-4: Furnishing & Rotation Script)..... 24
(Code 3-5: Landscape & UI Script)..... 25
(Fig 4-1 :72-Unit Automated Cluster Layout)..... 27
(Fig-4-2: Automated Infrastructure and Urban Grid) 28
(Fig 4-3: Automated 72-Unit Residential Cluster)..... 29
(Fig 4-4: Automated Site Components Distribution) 30
(Fig 4-5-a: Final Landscape and Site Integration View) 31
(Fig 4-5-b: Detailed View of the 200m² Residential Unit)..... 32
(Fig 5-2 :72-Unit Automated Cluster Layout)..... 35
(Fig 5-3 :Procedural Street Furniture and Site Details)..... 36

Abstract

This research explores the role of visual programming within the Dynamo environment in automating the design processes for smart residential complexes. The study focuses on developing an advanced programming algorithm in Python to generate a prototype of a 200-square-meter residential unit, enabling iteration and automatic distribution to create an integrated residential complex. The research also analyzes design efficiency and addresses specific technical challenges, such as wall joints, ultimately aiming to anticipate future improvements to transform the model into a smart building system that reduces time and effort in large-scale projects.



Chapter One: Introduction

1.1 Overview of Architectural Modeling:

Architectural modeling has witnessed a significant transformation with the advent of digital technologies. Traditionally, architectural design relied on manual drafting, which was time-consuming and prone to errors. Today, Building Information Modeling (BIM) allows architects to create detailed 3D representations that integrate functional and physical characteristics of a building. This shift has not only improved accuracy but also paved the way for automation in design processes[3].

1.2 Technologies Used in the Evaluation:

This research integrates a suite of advanced programming and engineering technologies to ensure precision in modeling and evaluation:

- **Dynamo (Visual Programming Environment):**

  Utilized as the primary interface to bridge programming logic with architectural modeling. It enables the control of elements through nodes and visual scripts, facilitating parametric design workflows.

- **Python (Scripting Engine):**

  Integrated within Dynamo to develop custom algorithms and handle complex computational tasks that exceed the capabilities of standard nodes. It is specifically used for advanced geometry calculations and precise control over architectural element properties.

- **Revit (BIM Platform):**

  Serves as the core environment for visualizing final results and generating 3D models. It is used to evaluate the efficiency of automated distribution and verify the structural integrity of the generated units.

- **Algorithmic Data Processing Tools:**

  Specialized software techniques used to automatically evaluate and process "Wall Joints". This ensures the continuity of the structural model and eliminates the need for manual corrections. [1].

1.3 Problem Statement:

Traditional architectural design often suffers from manual errors and time inefficiency, particularly in large residential complexes. Designing and repeating units manually is tedious and prone to inaccuracies, especially regarding complex wall joints. Thus, there is a critical need for automated solutions to ensure higher precision and streamline the modeling process[3].

1.4 Research Objectives:

The primary goal is to develop an automated modeling system using Dynamo and Revit through specific objectives include:

1. Developing a Python-based algorithm for a (200m²) residential unit.
2. Automating unit replication within a residential complex.
3. Solving technical wall joint issues.
4. Reducing manual effort and design time.

1.5 Research Importance:

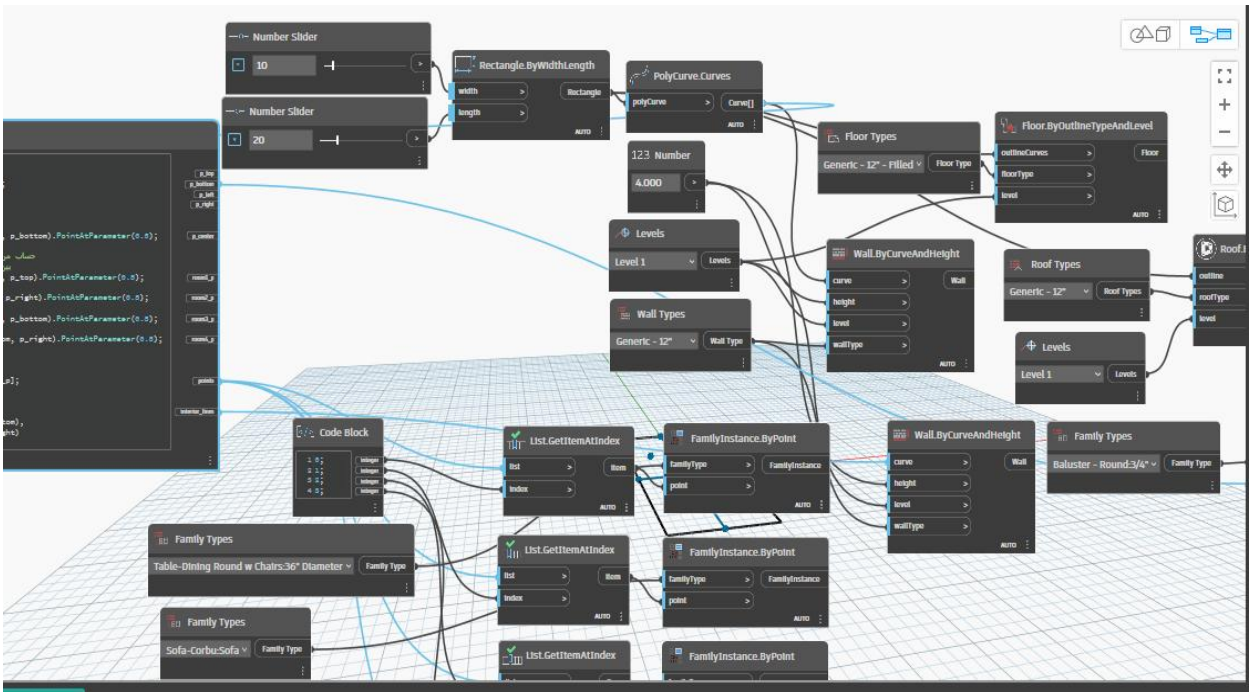
The significance of this research lies in its potential to revolutionize architectural modeling by bridging the gap between manual design and computational automation. By utilizing Dynamo and Revit 2026, this study provides a framework for:

1. Improving accuracy in complex structural detailing, such as wall junctions.
2. Significantly reducing the project delivery timeline through algorithmic automation.
3. Offering a scalable and flexible solution for high-density residential complexes.
4. Enhancing the efficiency of BIM-based workflows for architectural engineering[1].

Chapter Two: Theory Part

2.1 Introduction:

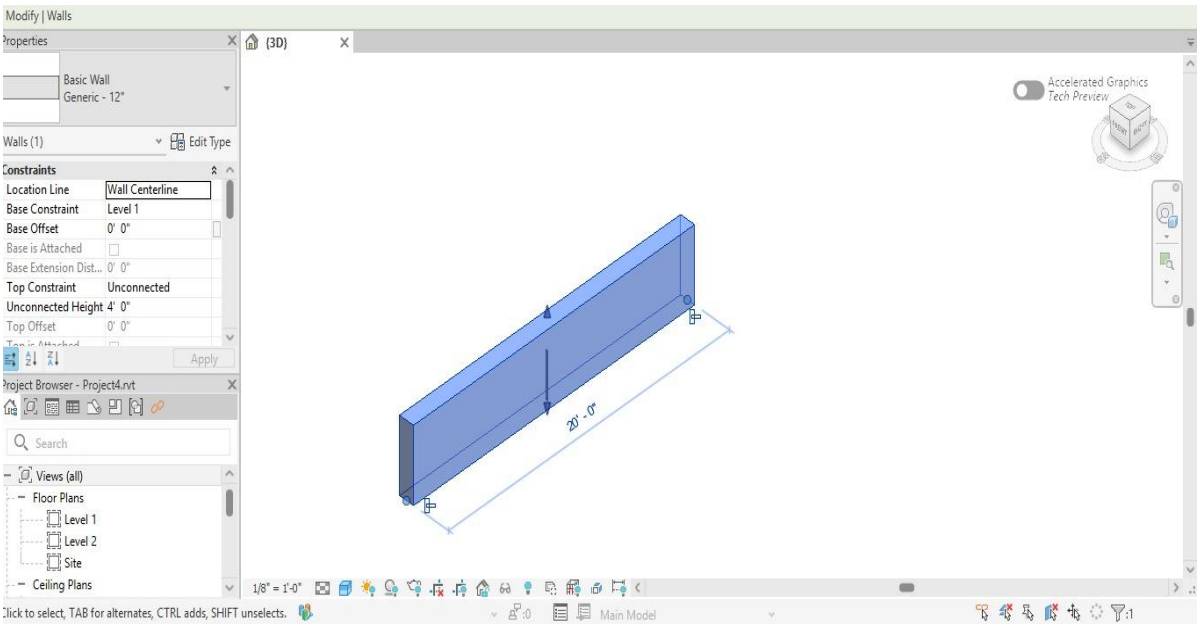
This chapter reviews the fundamental concepts and digital tools employed in the development of the automated residential complex. It focuses on the integration of BIM (Building Information Modeling) through Revit and the power of computational design via Dynamo. The chapter highlights how Python scripting acts as the primary engine for automating architectural elements, replacing manual drafting with algorithmic logic to ensure precision and efficiency in designing 200m² residential units[1].



(Fig-2-1: Automated BIM Modeling via Dynamo and Python)

2.2 Building Information Modeling (BIM) Concept:

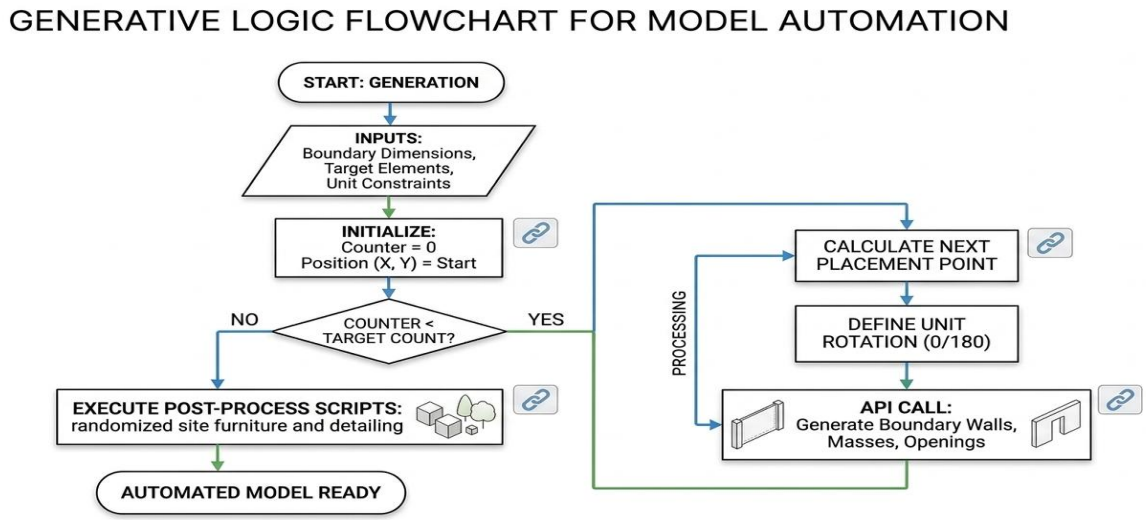
The core of this research is based on the BIM philosophy, where architectural elements are no longer just static lines but data-rich digital objects. As shown in (Fig-2-2), when a wall is selected within the Revit environment, the system immediately displays its "Constraints" and "Dimensions" in the Properties palette. This metadata (such as Base Offset, Top Constraint, and Unconnected Height) allows the Python script to interact with the wall not as a drawing, but as a programmable database object, enabling high-precision automated modelling[1].



(Fig-2-2: BIM Data-Driven Elements)

2.3 Algorithmic and Generative Design:

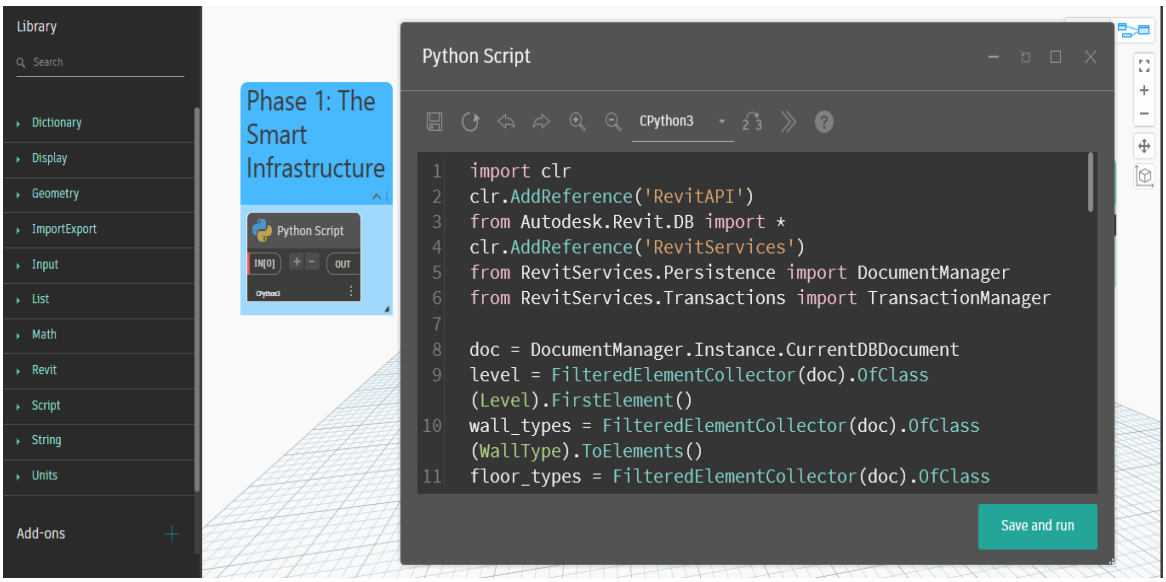
This research relies on algorithmic design philosophy, using logic and rules to generate the residential complex automatically. This approach allowed for the precise and rapid replication of 72 units, ensuring the uniform distribution of plots (10 \times 20 m) through computational loops. By shifting from "drawing" to "coding," the system can handle complex repetitions and spatial constraints in a way that exceeds the limits of traditional manual drafting[2].



(Fig-2-3: Generative Model Flowchart)

2.4 Visual Programming Environment (Dynamo) & API Integration:

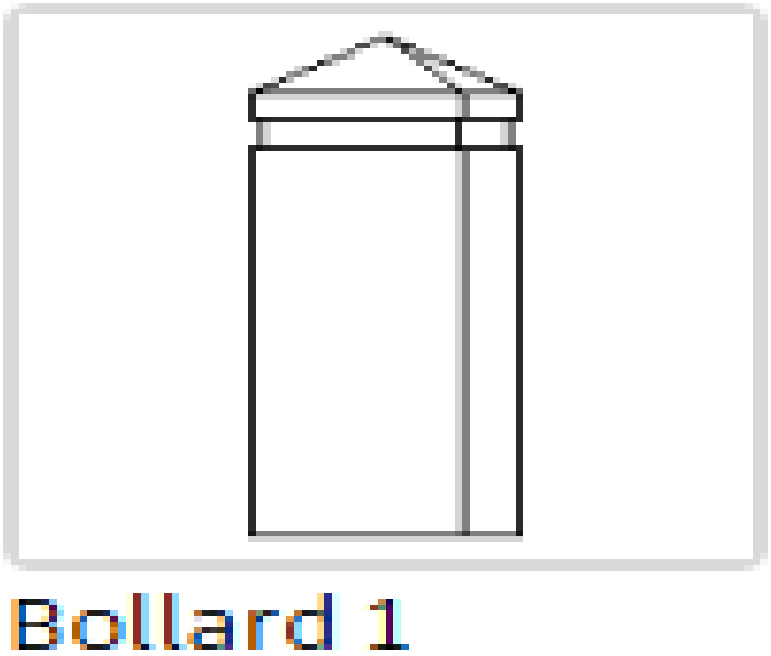
Dynamo serves as the computational engine that bridges the gap between Python scripts and the Revit environment. It provides a visual interface where logical nodes are connected to execute complex tasks. In this project, Dynamo was used as a host for Python Script nodes, allowing for the direct invocation of the Revit API. This integration enables the automation of repetitive architectural tasks, such as the generation of the residential cluster masses, the creation of parametric openings, and the randomized distribution of environmental elements, significantly reducing manual modeling time and proving the computational approach[2],[4].



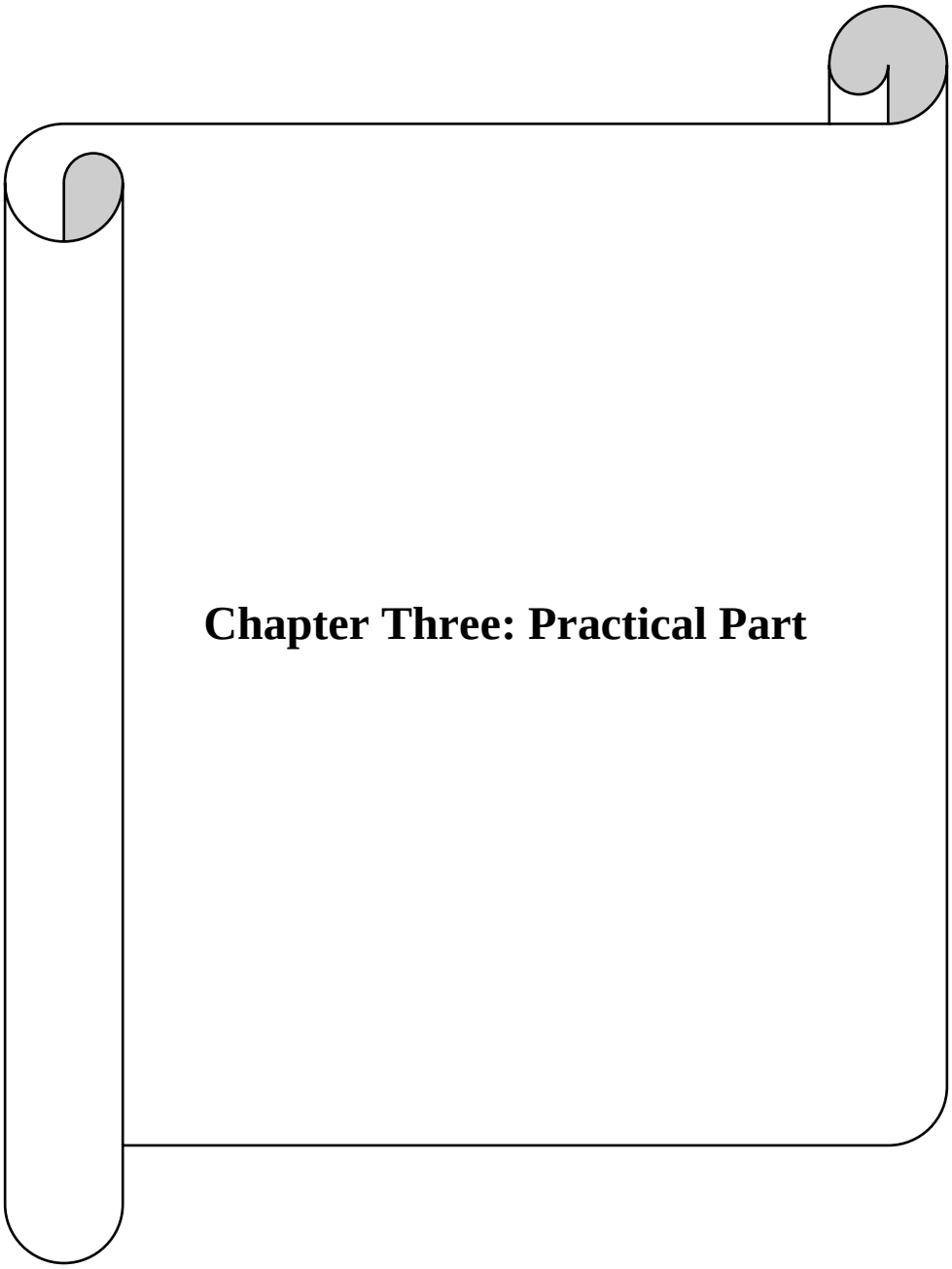
(Fig-2-4: Dynamo & Python API Integration)

2.5 Smart Infrastructure and Residential Complexes:

The research concludes by integrating Smart City systems, including programmed security gates (Bollards) and sustainable lighting with motion sensors to save energy. Additionally, each villa is linked to interactive metadata that displays floor plans and areas immediately upon interaction[4].



(Fig-2-5: Automated Security Bollard Component)



Chapter Three: Practical Part

3.1 Introduction:

This chapter details the practical implementation and the computational logic behind the automated generation of the smart residential complex. The methodology relies entirely on algorithmic workflows developed through Python scripting integrated within the Dynamo environment. It describes the technical stages of the project, starting from defining geometric parameters to the full automated execution of architectural elements in Revit 2026, ensuring that the entire design process is governed by code rather than manual modeling.

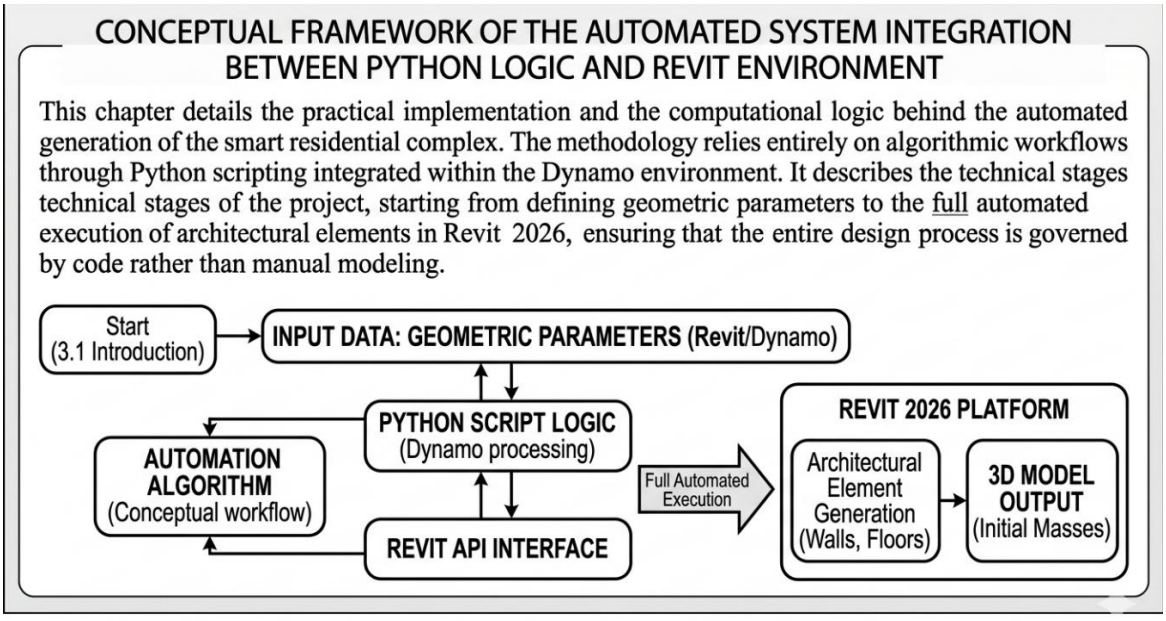
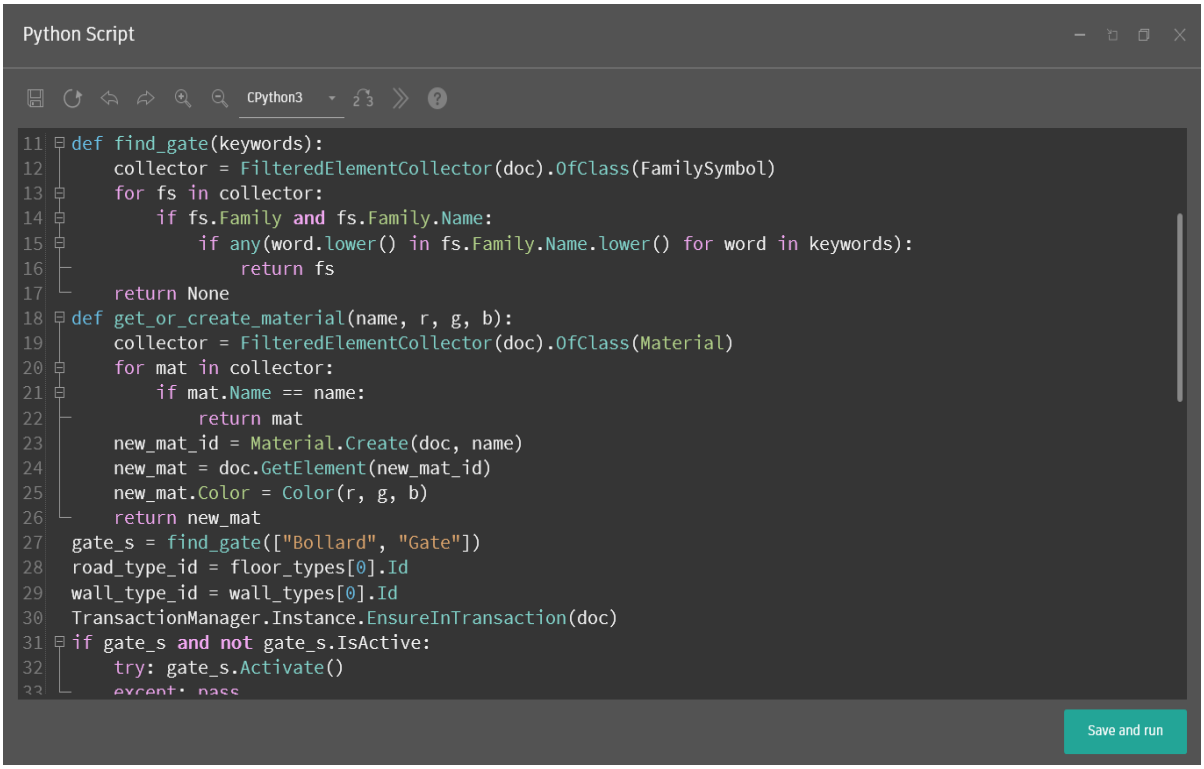


Fig. 3-1: Conceptual framework of the automated system integration.

3.2 Global Infrastructure & Grid Establishment:

This system is responsible for establishing the project's foundation, including the 200x200m boundary, main and secondary roads, and the smart security bollards. As shown in (Code 3-2), the script utilizes the FilteredElementCollector to programmatically identify Revit families and material types. It then applies logic to automate the generation of the urban grid, ensuring that all elements are placed with high precision based on coordinate parameters rather than manual input.



(Code 3-2: Infrastructure & Grid Script)

3.3 Automated Residential Cluster Generation:

This section details the algorithmic logic used to generate the residential units. As illustrated in (Code 3-3), the script employs Nested Loops to iterate through the main blocks, programmatically assigning walls, floors, and roofs. The Python logic is designed to automatically distinguish between different material types—such as brick walls and "Storefront" glass facades—by searching for specific keywords within the Revit database. This automated classification ensures that each unit is constructed with the correct architectural specifications without manual intervention.

Python Script

CPython3

```
    walls = FilteredElementCollector(doc).OfClass(WallType).ToElements()
    for w in walls:
        try:
            n = w.get_Parameter(BuiltInParameter.SYMBOL_NAME_PARAM).AsString().lower()
            if any(h.lower() in n for h in name_hints): return w
        except: continue
    return walls[0]
def find_floor_type(hints):
    types = FilteredElementCollector(doc).OfClass(FloorType).ToElements()
    for t in types:
        try:
            n = t.get_Parameter(BuiltInParameter.SYMBOL_NAME_PARAM).AsString().lower()
            if any(h.lower() in n for h in hints): return t
        except: continue
    return types[0]
glass_wt = find_wall(["Storefront", "Curtain"])
brick_wt = find_wall(["Brick", "Generic", "Basic"])
int_wt = find_wall(["100", "Interior"])
roof_type = find_floor_type(["Generic", "Concrete", "Roof"])
all_doors = FilteredElementCollector(doc).OfClass(FamilySymbol).OfCategory
(BuiltInCategory.OST_Doors).ToElements()
door_t = next((d for d in all_doors if "double" in d.Family.Name.lower()), all_doors[0])
```

Save and run

(Code 3-3: Residential Units Logic)

3.4 Procedural Urban Furnishing & Unit Details:

This module automates the placement of essential site components and internal unit details. As shown in (Code 3-4), the script utilizes a custom function get_family_symbol to programmatically retrieve and activate Revit families such as "Street Light", "Outdoor Bench", and "Beetle" cars. The algorithm handles the complex task of locating specific architectural elements, like revolving doors, by filtering parameters for exact dimensions (e.g., 84" x 96"). Furthermore, it ensures realistic alignment within the garages by managing the orientation of car instances, replacing the need for manual rotation and placement.

Python Script

CPython3

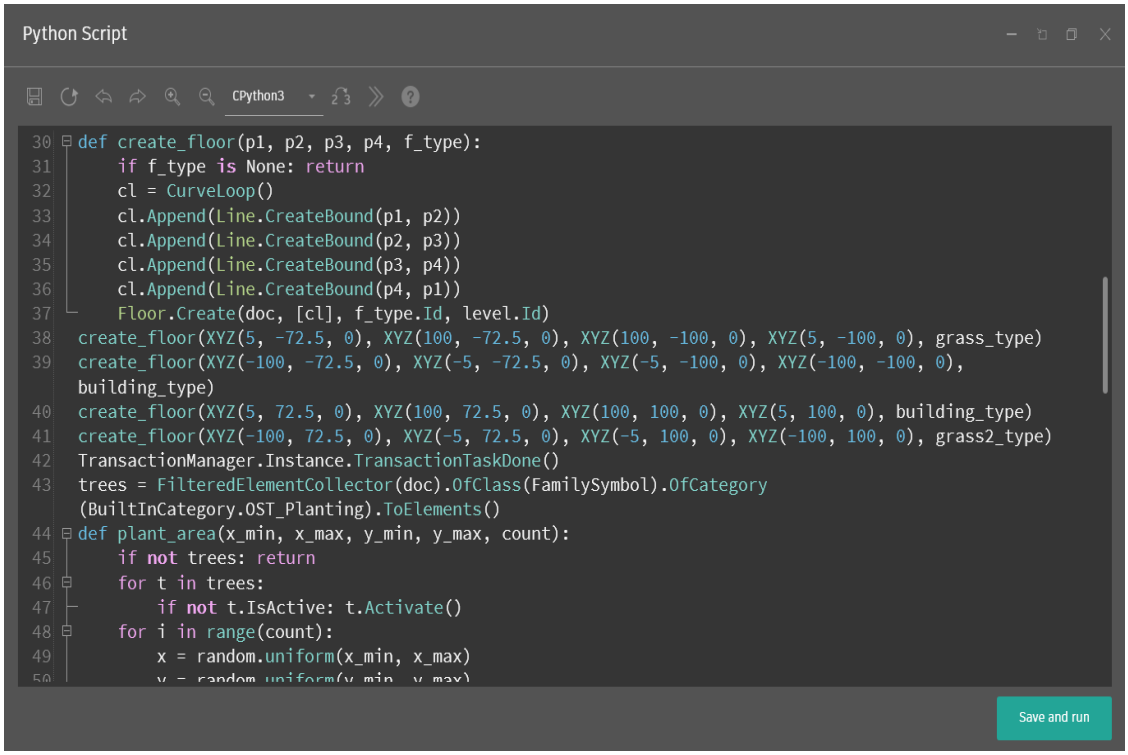
```
def get_family_symbol(family_name):
    collector = FilteredElementCollector(doc).OfClass(FamilySymbol)
    for sym in collector:
        if sym.Family.Name == family_name:
            if not sym.IsActive: sym.Activate()
            return sym
    return None
car_t = find_car("Beetle")
garage_type = find_floor_type(["Gray", "Concrete", "Generic"])
garden_type = find_floor_type(["Grass", "Green", "Generic"])
light_sym = get_family_symbol("Street Light - Standard")
bench_sym = get_family_symbol("Outdoor Bench")
door_symbols = FilteredElementCollector(doc).OfClass(FamilySymbol).OfCategory
(BuiltInCategory.OST_Doors)
target_door = None
for ds in door_symbols:
    fam_name = ds.Family.Name
    type_name = ds.get_Parameter(BuiltInParameter.SYMBOL_NAME_PARAM).AsString()
    if "Revolving" in fam_name and "84" in type_name and "96" in type_name:
        target_door = ds
        break
TransactionManager.Instance.EnsureInTransaction(doc)
if car_t and not car_t.IsActive: car_t.Activate()
```

Save and run

(Code 3-4: Furnishing & Rotation Script)

3.5 Stochastic Landscaping & Advanced System UI:

To simulate a natural environment, the system implements a Stochastic Algorithm for tree distribution. As shown in (Code 3-5), the function plant_area utilizes the random.uniform method to generate varied coordinates, ensuring that trees are placed in non-repetitive, natural patterns within defined boundaries. Additionally, the script handles the automated creation of complex floor geometries through the create_floor function, which defines precise CurveLoop boundaries based on XYZ coordinates. This computational approach replaces manual landscaping, providing a professional and randomized aesthetic within the Revit environment.



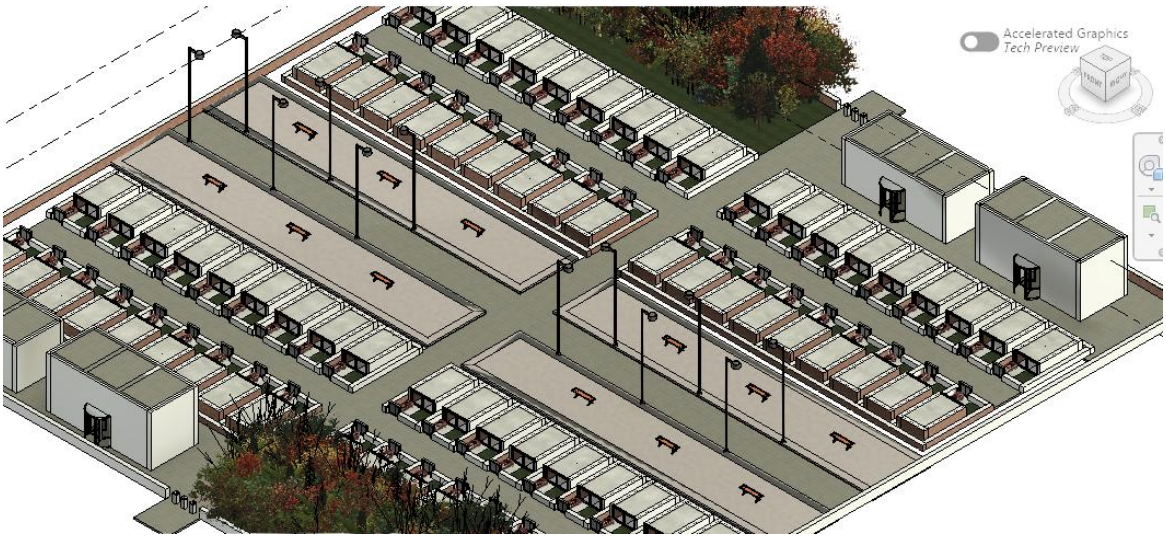

```
30 def create_floor(p1, p2, p3, p4, f_type):
31     if f_type is None: return
32     cl = CurveLoop()
33     cl.Append(Line.CreateBound(p1, p2))
34     cl.Append(Line.CreateBound(p2, p3))
35     cl.Append(Line.CreateBound(p3, p4))
36     cl.Append(Line.CreateBound(p4, p1))
37     Floor.Create(doc, [cl], f_type.Id, level.Id)
38 create_floor(XYZ(5, -72.5, 0), XYZ(100, -72.5, 0), XYZ(100, -100, 0), XYZ(5, -100, 0), grass_type)
39 create_floor(XYZ(-100, -72.5, 0), XYZ(-5, -72.5, 0), XYZ(-5, -100, 0), XYZ(-100, -100, 0),
   building_type)
40 create_floor(XYZ(5, 72.5, 0), XYZ(100, 72.5, 0), XYZ(100, 100, 0), XYZ(5, 100, 0), building_type)
41 create_floor(XYZ(-100, 72.5, 0), XYZ(-5, 72.5, 0), XYZ(-5, 100, 0), XYZ(-100, 100, 0), grass2_type)
42 TransactionManager.Instance.TransactionTaskDone()
43 trees = FilteredElementCollector(doc).OfClass(FamilySymbol).OfCategory
   (BuiltInCategory.OST_Planting).ToElements()
44 def plant_area(x_min, x_max, y_min, y_max, count):
45     if not trees: return
46     for t in trees:
47         if not t.IsActive: t.Activate()
48     for i in range(count):
49         x = random.uniform(x_min, x_max)
```

(Code 3-5: Landscape & UI Script)

Chapter Four: Results and Discussion

4.1 Introduction:

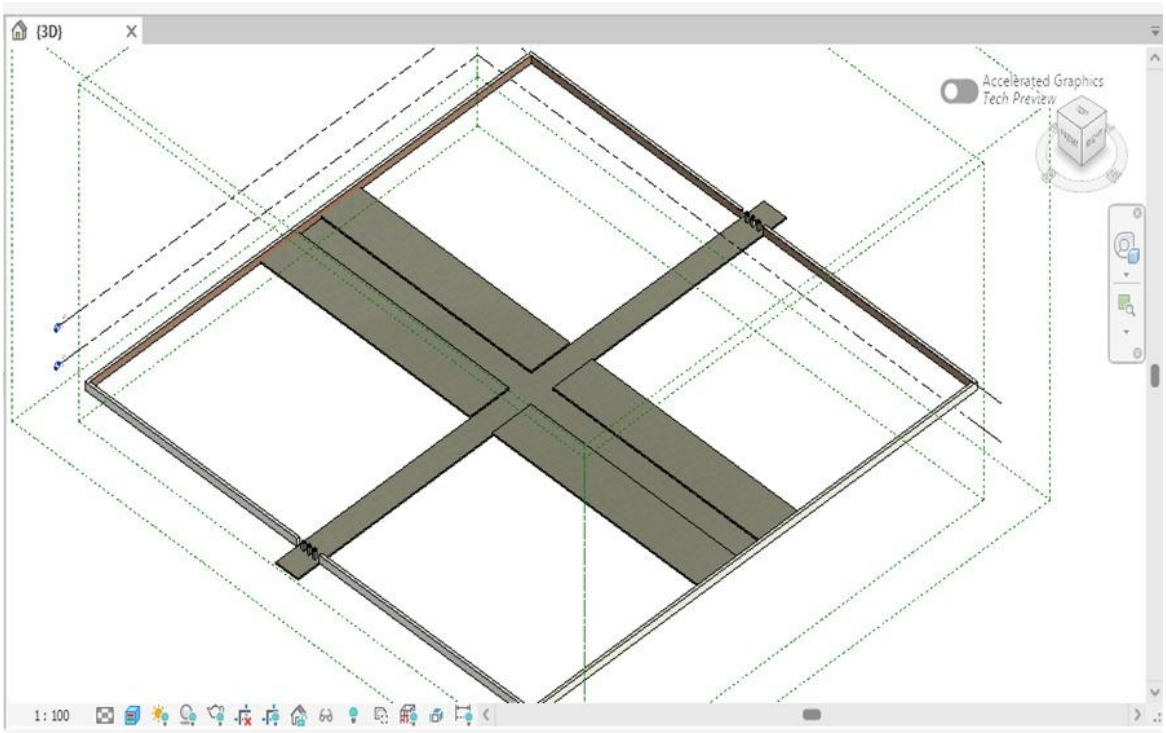
This chapter presents the final outcomes of the automated design system and evaluates its performance in generating the smart residential complex. The results demonstrate the system's ability to execute a comprehensive architectural workflow, ranging from the structural generation of the building masses to the automated furnishing and site detailing of each unit. By analyzing the final model, this section discusses how the integration of Python and Dynamo achieved a high level of technical and architectural accuracy, providing a precise digital representation of the complex where every element—from the main villa walls to the smallest site components—is governed by the established algorithmic logic.



(Fig 4-1 :72-Unit Automated Cluster Layout)

4.2 Infrastructure and Grid Execution:

This section illustrates the practical execution of the infrastructure logic. The automated workflow successfully established the project's boundary fence, the main and secondary road networks, and the strategic placement of security bollards. This precise grid layout was generated through the algorithmic parameters defined in (Code 3-2), where the system ensures that each residential unit is accurately allocated within its designated coordinates as shown in (Fig 4-2).



(Fig-4-2: Automated Infrastructure and Urban Grid)

4.3 Automated Residential Units Generation:

This section demonstrates the implementation of the core architectural logic for the residential units. Based on the algorithmic parameters established in (Code 3-3), the system successfully generated the structural elements, including the main walls and roofs for each unit. The workflow maintains the strict spatial requirement of 200m² per unit, ensuring architectural consistency and precision across the entire cluster as shown in (Fig 4-3).



(Fig 4-3: Automated 72-Unit Residential Cluster)

4.4 Urban Furnishing and Site Detailing Execution:

This final stage of execution demonstrates the system's ability to automate complex site detailing and furnishing. Based on the logic defined in (Code 3-4), the workflow successfully populated the site with essential urban elements, including street lighting, public benches, and precise pavement textures. Furthermore, the results highlight the automated placement of vehicles within the fully enclosed garages, ensuring that every architectural requirement was met with high precision as shown in (Fig 4-4).



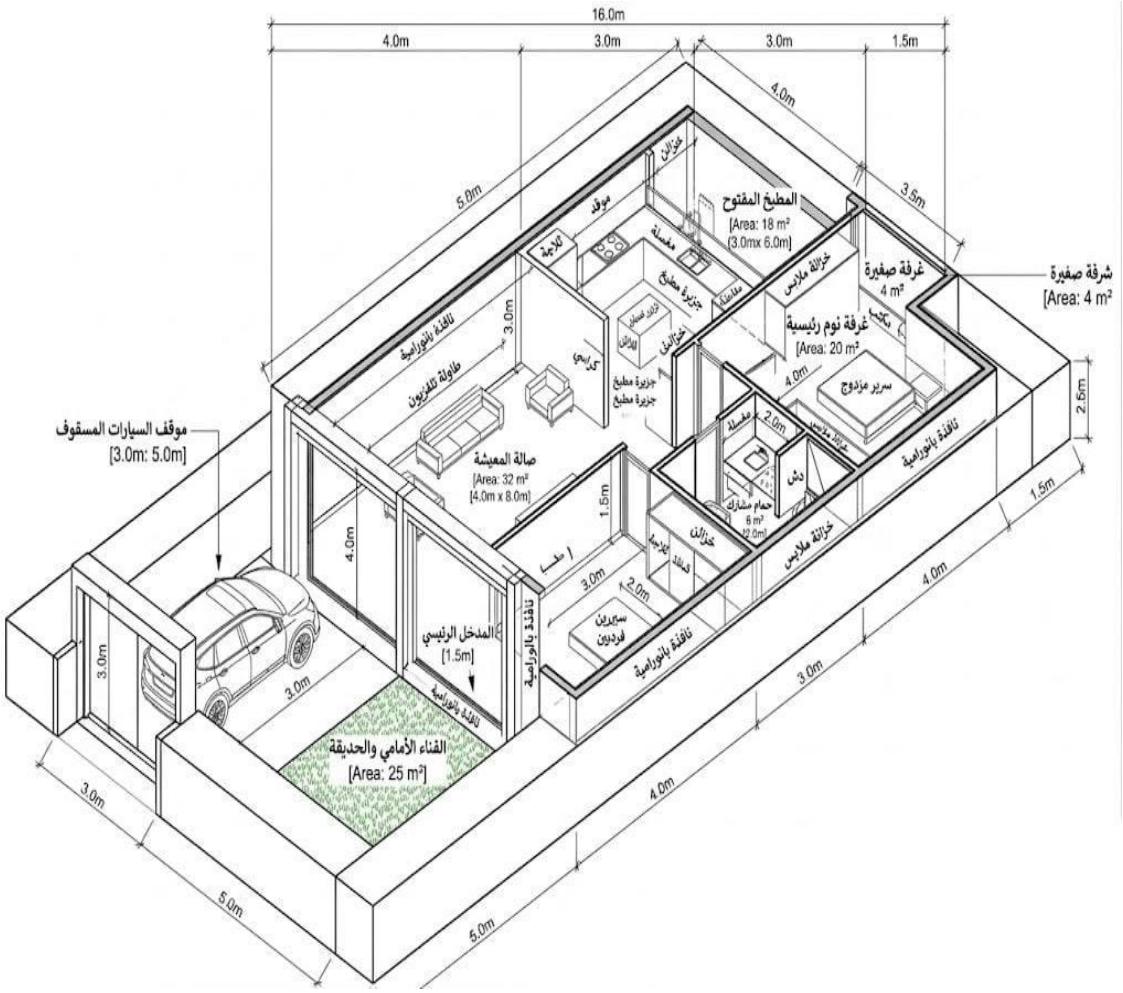
(Fig 4-4: Automated Site Components Distribution)

4.5 Comprehensive Final Model Visualization:

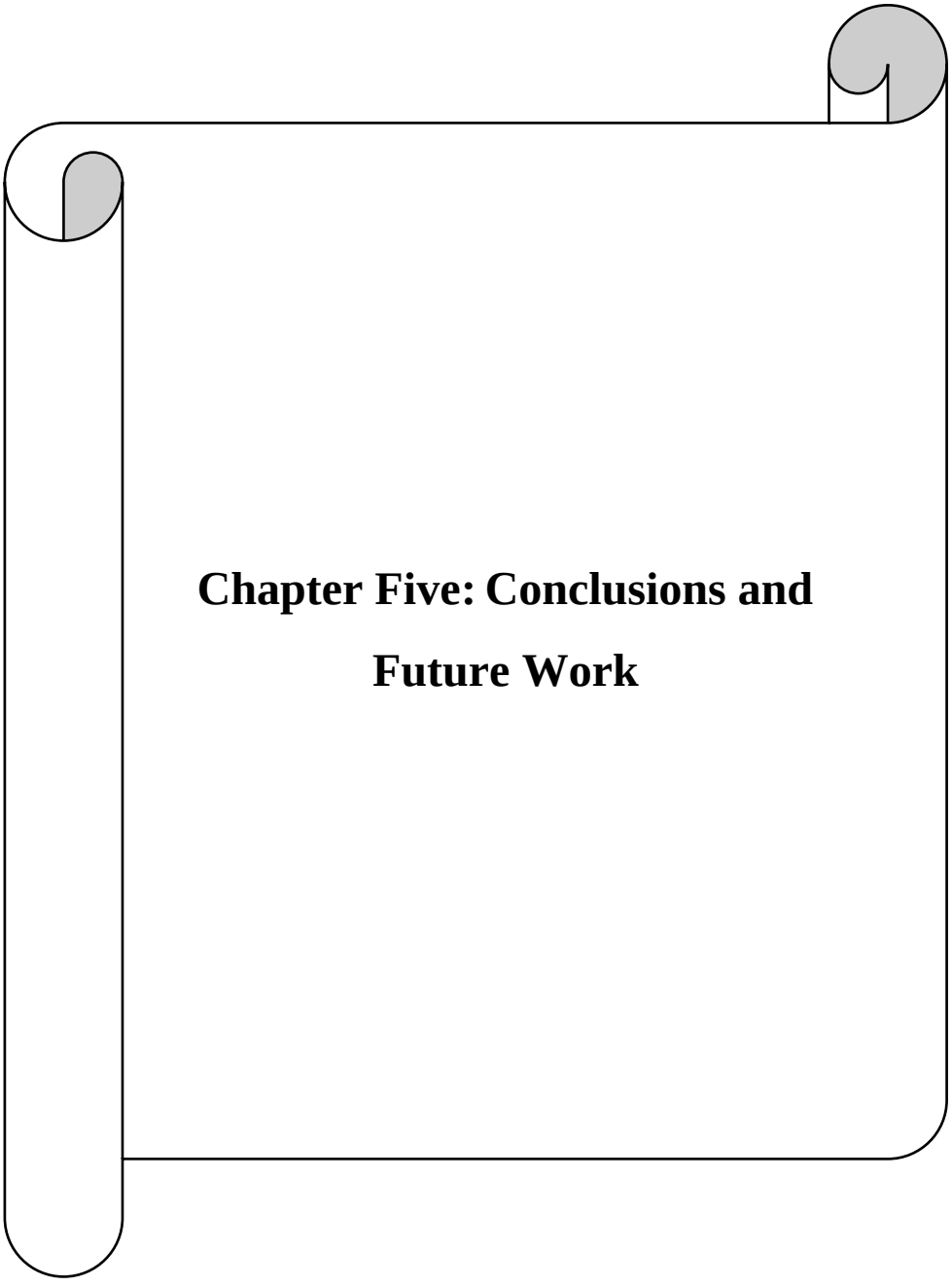
This final stage of execution provides a comprehensive visualization of the integrated architectural code. Based on the logic defined in (Code 3-5), the system generated two distinct outcomes to verify the design logic: the first showcasing the final landscape integration, and the second providing a detailed axonometric breakdown of the interior spaces. This automated process confirms the successful generation of the 200m² residential units and the specialized garage entrances as shown in (Fig 4-5-a, b).



(Fig 4-5-a: Final Landscape and Site Integration View)



(Fig 4-5-b: Detailed Axonometric View of the 200m² Residential Unit)



Chapter Five: Conclusions and Future Work

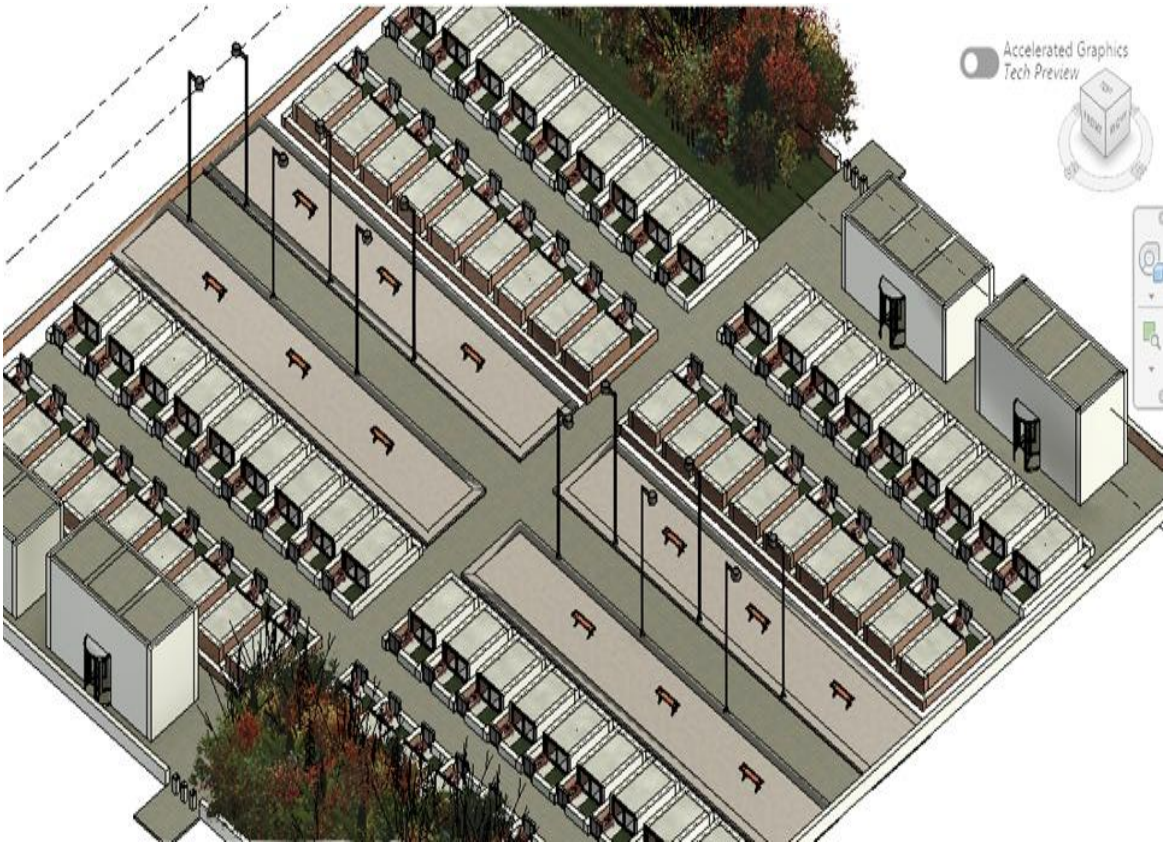
5.1 Introduction:

This final chapter serves as the culmination of the research and development process for the automated smart residential complex. After establishing the theoretical framework and developing the specialized Python-based modules in the preceding chapters, this section provides a comprehensive evaluation of the overall project outcomes. The primary focus here is to synthesize the individual results of the infrastructure, unit generation, and urban furnishing modules into a singular, cohesive architectural conclusion.

Furthermore, this chapter highlights how the integration of computational logic via Revit and Dynamo has successfully addressed the complex challenges of modern urban design, particularly in achieving spatial precision for the 200m² residential units. By reflecting on the efficiency gains and the seamless coordination of diverse architectural elements, this introduction paves the way for a detailed summary of the project's workflow and integrated results. Finally, it explores the future potential of this algorithmic approach, suggesting how it can be scaled and enhanced to meet the evolving demands of smart city development.

5.2 Full Project Workflow:

This section encapsulates the complete computational workflow applied to the residential complex. By integrating multiple Python modules, the project achieved a holistic architectural solution that covers structural generation, unit distribution, and site furnishing. The final model represents a seamless blend of automated precision and architectural design, where every element—from the main villa walls to the smallest street lamp—was placed through algorithmic logic.



(Fig 5-2 :72-Unit Automated Cluster Layout)

5.3 Integrated Results & Visualization:

The execution resulted in a cohesive 72-unit cluster. The visualization demonstrates the successful coordination between individual unit details, such as the automated garages and private entrances, and the overall urban fabric, including street benches and lighting. This integration ensures that the complex is not just a collection of buildings, but a functional and aesthetically unified environment.



(Fig 5-3 :Procedural Street Furniture and Site Details)

5.4 Future Work & Development:

Looking ahead, this project opens the door for several advancements:

- **Dynamic Scaling:** Developing the script to adapt to different site topographies and irregular boundary shapes.
- **AI Integration:** Utilizing machine learning to suggest the most efficient unit layouts based on historical design data.
- **Sustainability Optimization:** Enhancing the code to automatically calculate and adjust window-to-wall ratios for better energy efficiency.

6 References

[1] Autodesk Revit API Developers Guide. Official Documentation.

[2] Standard Python Library Documentation. Python Software Foundation.

[3] Eastman, C., et al. (2011). BIM Handbook: A Guide to Building Information Modeling.

[4] Relevant Online Programming Communities (e.g., RevitForum, Stack Overflow, Dynamo BIM).

الخلاصة

يستعرض هذا البحث دور البرمجة المرئية عبر بيئة **(Dynamo)** في أتمتة عمليات التصميم للمجمعات السكنية الذكية. تركز الدراسة على تطوير خوارزمية برمجية متقدمة بلغة **(Python)** لتوليد نموذج أولي لوحدة سكنية بمساحة 200 متر مربع، مع تفعيل خاصية التكرار والتوزيع التلقائي لإنشاء مجمع سكني متكامل. كما يتطرق البحث إلى تحليل كفاءة التصميم ومعالجة التحديات التقنية الدقيقة مثل **(Wall Joints)**، وصولاً إلى استشراف التحسينات المستقبلية لتحويل النموذج إلى نظام بناء ذكي يهدف إلى تقليص الوقت والجهد في المشاريع الكبرى.

جمهورية العراق
وزارة التعليم العالي والبحث العلمي
جامعة بابل
كلية العلوم للبنات
قسم علوم الحاسوب

أتمتة النمذجة المعمارية
لزيادة الإنتاجية باستخدام (داينمو) و(ريفيت)

بحث مقدم من قبل الطالبة

سجى حيدر عبد الزهرة

إلى مجلس كلية العلوم للبنات - قسم علوم الحاسوب / جامعة بابل، وهي جزء من متطلبات نيل درجة البكالوريوس في علوم الحاسوب.

بإشراف

الدكتورة سحر عادل كاظم

١٤٤٧ مـ
٢٠٢٦ هـ